Software Requirements Specification

Aurelia Care — Doctor Appointment Booking System

Version 1.0 | Academic implementation baseline

Document details	Value
Prepared by	Ronit Rajput
Roll No.	E052
Program	Computer Science and Business Systems
Division / Batch	E / 2
Date of preparation	26 October 2026
Date of submission	_____
Grade / faculty review	_____
Repository reference	89168e01cc9c8133747525465334d0e14a2b4353
Approval status	Prepared for academic review; faculty approval not recorded

Format and lab-manual alignment

Experiment 1, Part B, curiosity question 2 asks students to use a sample SRS format and prepare the relevant parts. The manual does not supply a mandatory SRS template. This document uses an IEEE 830-style organization: Introduction, Overall Description, Specific Requirements, and supporting appendices. It retains the manual's student-details fields; it does not claim standards certification or that an external organization's template was downloaded.

Document control

Version	Date	Change
1.0	26 October 2026	Initial SRS aligned with the implemented scope and the corrected custom-port configuration.

Contents

- 1. Introduction: purpose, scope, definitions, references, conventions.
- 2. Overall description: users, environment, constraints, assumptions, context.
- 3. Specific requirements: interfaces, functions, data, quality, and business rules.
- Appendix A. Core use cases and exception flows.
- Appendix B. Acceptance criteria and requirements traceability.
- Appendix C. Feasibility, requirement reconciliation, development plan, and open targets.

1. Introduction

1.1 Purpose

This SRS defines the required behaviour, interfaces, data constraints, and quality expectations of Aurelia Care. It provides a common reference for the student developer, faculty reviewer, tester, and demonstration users. Requirements describe observable outcomes; implementation notes explain the selected academic baseline.

1.2 Problem statement

Manual appointment coordination can leave doctor availability unclear, duplicate slot reservations, and scatter booking history across separate records. The proposed system shall provide one role-controlled interface for viewing schedules, reserving appointments, and maintaining consistent appointment records.

1.3 Product scope

- Patients register, authenticate, search active doctors, inspect future slots, book consultations, and manage eligible appointments.
- Doctors publish availability, view their own appointments and assigned patients, update profiles, and complete eligible consultations.
- Administrators manage doctors, specialties, accounts, appointments, reports, and validated Excel exchange.
- SQLite maintains authoritative records; the interface displays confirmations and notifications. Payments are explicitly simulated.

1.4 Out of scope

Real payment processing, live email/SMS delivery, prescriptions, diagnostics, laboratory orders, receptionist workflows, clinical advice, teleconsultation, public production hosting, and healthcare compliance certification are outside this baseline. Character names and portraits are fictional demonstration presentation assets.

1.5 Terms and abbreviations

Term	Meaning
SRS / FR / NFR	Software Requirements Specification / Functional Requirement / Nonfunctional Requirement
API / JSON	Application Programming Interface / JavaScript Object Notation
JWT / session	Signed authentication token / revocable server-side login record
Slot	A doctor’s dated start–end availability interval
Confirmed appointment	A persisted reservation with a successful simulated payment record
Snapshot	Data exported at a particular time; not a live Excel connection
IST	Asia/Kolkata time zone, UTC+05:30

1.6 Conventions

“Shall” identifies a baseline requirement. FR, DR, NFR, and BR identifiers support traceability. Priority H means necessary for the core workflow; M means supporting functionality. Future goals are marked TARGET and are not represented as verified capabilities. This SRS documents the present academic scope, not a new production-service contract.

2. Overall Description

2.1 Product perspective

Aurelia Care is a browser-based, role-controlled application. The React interface communicates with a FastAPI service through HTTP/JSON. The service validates requests, authorizes actions, and reads or updates SQLite. The browser does not directly open the database file.

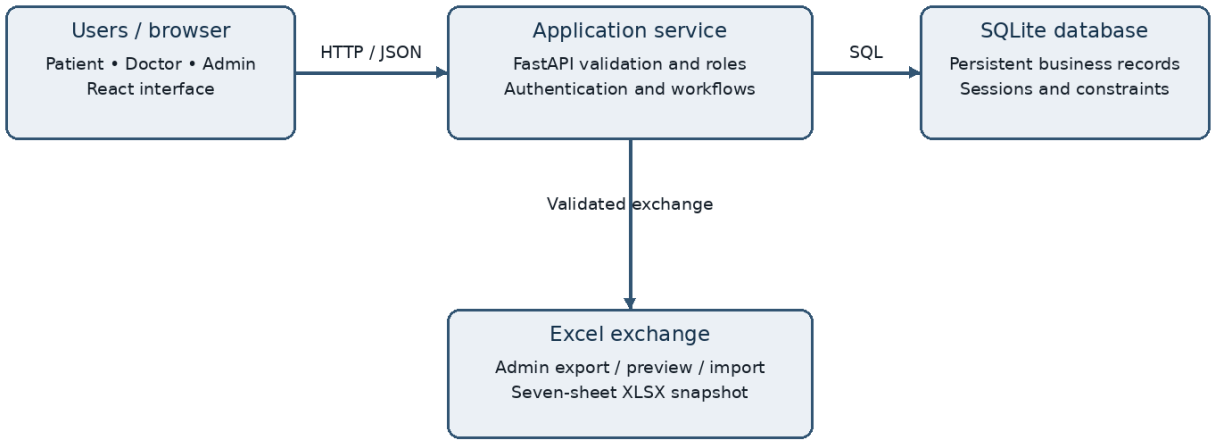


Figure 1. Logical system context and data exchange boundaries.

2.2 User classes

User class	Responsibilities / access
Visitor	View public doctor information; register as a patient; sign in.
Patient	Access own profile, appointments, and notifications; perform permitted booking changes.
Doctor	Access own profile, availability, consultations, and assigned appointment information.
Administrator	Manage permitted accounts, doctors, specialties, appointments, statistics, and workbook exchange.

2.3 Operating environment

The demonstration uses Python 3.12+, Node.js 22+, React/TypeScript/Vite, FastAPI/Pydantic/Uvicorn, and SQLite. The production frontend build and API are served by one application process. Vite can provide an optional development server. Linux execution is verified; a Mac Bash launcher is supplied, while native Mac and Windows execution are not claimed as tested.

2.4 Constraints and dependencies

- Use SQLite as the authoritative datastore and XLSX as explicit administrative exchange.
- Use one application process for the SQLite demonstration; persistent storage is necessary to retain data.
- Initial dependency installation requires network access. Bundled application fonts and portraits require no external image host during normal operation.
- A future public installation requires HTTPS, explicit trusted origins, a protected signing secret, and a persistent DB_PATH.

2.5 Assumptions

The host clock is accurate; users have an appropriate browser and authorized credentials; the administrator maintains doctor data; and patients understand that the demonstration does not provide emergency care or real payment processing. Users refresh or repeat a request to obtain newer data; no server-push synchronization is specified.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User interface

ID	Requirement	Verification
UI-01	The system shall provide public home, doctor listing/profile, login/registration, and role dashboards.	Navigate the main pages and verify role-specific content.
UI-02	Forms shall have visible labels, required-field feedback, and understandable success/error states.	Submit valid and invalid inputs.
UI-03	Layouts shall adapt to desktop and mobile viewports; core navigation and booking controls shall remain usable.	Inspect desktop and 390-pixel mobile layouts.
UI-04	Booking shall show selected doctor, date/time, fee, and simulated-payment context before confirmation.	Inspect the review dialog and final confirmation.
UI-05	The interface shall support keyboard focus, a skip link, and reduced-motion preferences in the implemented interactions.	Inspect focus, keyboard navigation, and reduced-motion rendering.

The UI applies consistency, user control, and reduced memory load. A complete accessibility certification or usability study is not an acceptance claim.

3.1.2 Software and communication interfaces

The UI shall call relative /api endpoints using JSON, with the session cookie included for same-origin requests. Excel uploads shall use multipart form data; export shall return an XLSX attachment. Public hosting shall use HTTPS; local demonstration HTTP is supported.

Representative interface	Purpose
POST /api/auth/register; POST /api/auth/login	Patient registration and user authentication
GET /api/auth/me; POST /api/auth/logout	Session identity and logout
GET /api/doctors; GET /api/doctors/{id}/availability	Doctor search and available slots
POST /api/appointments	Create a booking
PATCH /api/appointments/{id}/cancel or /reschedule	Permitted changes
GET /api/admin/excel/export; POST .../validate or .../import	Administrative workbook exchange

3.1.3 Response conventions

Endpoints shall use 200/201 for successful operations as appropriate, 401 for missing/invalid authentication, 403 for unauthorized role or untrusted-origin actions, 404 for inaccessible/missing resources, 409 for conflicts, 422 for invalid inputs or rules, 402 for declined simulated payment, and 429 for the implemented authentication-rate limit. Authorization and resource ownership determine whether an inaccessible item is reported as not found.

3.2 Functional Requirements

3.2.1 Identity, authentication, and roles

ID / priority	Requirement	Acceptance check
FR-01 / H	The system shall register a patient using name, unique email, phone, and password; it shall create the linked patient profile.	Valid registration creates one user/profile pair; duplicate email is rejected.
FR-02 / H	The system shall accept valid credentials only for an active account and shall return a generic rejection for wrong, unknown, or inactive accounts.	Correct login succeeds; each invalid account condition is rejected.
FR-03 / H	The system shall create an eight-hour signed-cookie session with a server-side session record; logout shall revoke that session.	Authenticated account request succeeds; replay after logout returns 401.
FR-04 / H	The system shall enforce roles and record ownership on the backend, independently of interface visibility.	Patient admin access and access to another user’s appointment are denied.
FR-05 / M	The system shall allow authenticated users to edit supported own-profile contact fields and patients to edit profile notes.	Changed fields persist and are returned on retrieval.

3.2.2 Doctor discovery

ID / priority	Requirement	Acceptance check
FR-06 / H	The system shall list active doctors and support name search, specialty/date filtering, and fee/experience sorting.	Matching filters return suitable records in the requested order.
FR-07 / H	The system shall display doctor specialty, qualifications, experience, consultation fee, biography, and demonstration portrait.	Profile values correspond to stored doctor data.
FR-08 / H	The system shall display only available future slots for active doctors in public availability responses.	Past, blocked, booked, and inactive-doctor slots are excluded.

Input rules

Registration names shall be 2–100 characters, passwords 8–128 characters, and emails valid and unique without case-sensitive duplication. Login requires a nonempty password up to 128 characters. Phone input shall follow the implemented 7–20-character pattern allowing digits, plus, whitespace, parentheses, and hyphens. The API shall reject unsupported extra request fields.

3.2.3 Appointment workflow

ID / priority	Requirement	Acceptance check
FR-09 / H	Only an authenticated patient shall create a booking through the booking API using a slot identifier and simulated payment outcome.	Patient succeeds with valid input; anonymous and nonpatient requests are rejected.
FR-10 / H	Booking shall recheck the doctor’s active status, slot availability, and strictly future start time inside the write transaction.	Occupied/past/inactive-doctor slots cannot be booked; time equality is rejected.
FR-11 / H	On successful simulated payment, the system shall atomically save a confirmed appointment, mark the slot booked, save its payment, and notify patient and doctor.	All records agree after commit; injected failure leaves no partial writes.
FR-12 / H	A failed simulated payment shall not create a confirmed booking or reserve the slot; a later retry may succeed if the slot remains available.	Failure returns 402 without persistent business-record changes; successful retry creates one booking.
FR-13 / H	Patients or administrators shall cancel eligible confirmed appointments; cancellation shall release the slot and mark the simulated payment refunded.	Eligible cancellation changes all related states together.
FR-14 / H	Patients or administrators shall reschedule an eligible confirmed appointment to a future available slot of the same doctor atomically.	New slot is reserved, old slot released, appointment remains confirmed; conflict leaves original booking intact.

Appointment decision sequence

Authenticate → select/review slot → recheck availability within the transaction → evaluate simulated payment → create appointment/payment/notifications and reserve slot → commit → show confirmation. A declined payment, conflict, or persistence failure shall not leave a partially confirmed booking.

Stored states

Entity	Allowed states and meaning
Appointment	confirmed → cancelled or completed; rescheduling retains confirmed
Availability	available, booked, blocked
Payment	simulated_paid or refunded; method is simulation
Notification	unread or read

Search, review, and attempted payment are transient workflow steps, not extra persisted appointment statuses. Completion does not reopen the slot; the unique-slot constraint applies to noncancelled appointments.

3.2.4 Doctor and administrator functions

ID / priority	Requirement	Acceptance check
FR-15 / H	Doctors shall view only their own appointments and associated patient details; patients shall view their own appointment history.	Listings and direct-resource requests obey ownership.
FR-16 / H	A doctor shall publish future start–end slots without overlap; an identical previously blocked interval may be reactivated.	Valid interval is added/reactivated; overlap and invalid time order are rejected.
FR-17 / H	A doctor shall remove only their own unbooked slots from public availability; booked slots shall not be removed.	Removal marks an unbooked slot blocked; booked-slot removal returns a conflict.
FR-18 / M	Doctors shall edit their supported profile fields; an assigned doctor or administrator shall complete a confirmed consultation only after its start time has arrived.	Profile persists; early or unauthorized completion is rejected.
FR-19 / H	Administrators shall add/edit doctor profiles, manage specialties, and deactivate nonadministrator accounts. A referenced specialty shall not be deleted.	Valid management changes persist; invalid references and privileged-account restrictions are enforced.
FR-20 / H	Administrators shall reset nonadministrator passwords, reactivate those accounts, and revoke their existing sessions; activation changes shall also revoke sessions.	Old sessions fail after account change; new credentials work when active.
FR-21 / M	Administrators shall view appointment records and dashboard counts, booking-status totals, upcoming counts, doctor-wise counts, and creation trends.	Displayed summaries reconcile with stored records.

Doctor profile validation

Experience shall be an integer from 0 to 70; consultation fee shall be finite and between INR 0 and INR 100,000. Specialty length shall be 2–80 characters, qualification 2–150, and biography 10–3,000. Slot inputs shall use whole-minute local times without embedded time-zone offsets.

Administration boundaries

Public registration shall not create administrator or doctor accounts. Administrator accounts shall not be deactivated or reset through the nonadministrator account-management endpoints. Doctor credentials are managed through authorized administrative operations; generated installation passwords are not part of this SRS.

3.2.5 Notifications and Excel exchange

ID / priority	Requirement	Acceptance check
FR-22 / M	The system shall store in-app booking/change notifications for relevant users, list only the current user’s notifications, and allow that user to mark their own notifications read.	Correct notices appear; another user cannot mark them read.
FR-23 / M	When a patient requests notifications, the system shall generate a nonduplicated reminder for confirmed appointments within the next 24 hours.	Qualifying reminder appears once; no external delivery is implied.
FR-24 / H	An administrator shall export exactly seven business-data sheets from a consistent current database snapshot; credentials, sessions, and account-active flags shall be excluded.	Workbook names/headers and data match the export schema without secret columns.
FR-25 / H	An administrator shall validate and preview an uploaded workbook without committing changes; preview shall return counts and up to five records per sheet.	Valid preview is shown and database snapshot remains unchanged.
FR-26 / H	An administrator shall apply an import only after validation against current records; all merges shall commit together or roll back together.	Valid rows merge by ID; invalid data causes no partial update.

Workbook rules

- Sheets shall be Users, Patients, Doctors, Availability, Appointments, Payments, and Notifications, with exact exported headers.
- Missing workbook rows shall not delete database records. Existing ownership and account roles shall not be reassigned. Administrator records shall not be created or changed by import.
- New patient/doctor accounts shall have matching profiles and remain inactive until an administrator sets a password.
- The importer shall reject inconsistent references, overlapping slots, contradictory booking/payment states, formulas, and macros. Completed/cancelled appointment records shall not be rewritten.
- Limits shall be enforced: at most 5 MiB uploaded data, 200 archive members, 30 MiB uncompressed archive content, and 10,000 data records per worksheet.
- Exports shall escape formula-like text. Dates shall use YYYY-MM-DD text, times HH:MM text, and phone numbers text.

The workbook is not a continuously synchronized database. Startup/export/import may refresh the stored workbook snapshot; after ordinary web bookings the administrator shall export again to obtain current data.

3.3 Logical Data Requirements

ID / entity	Principal attributes	Relationship / constraint
DR-01 Users	user_id, name, email, phone, role, created_at, password_hash, active	Primary key user_id; unique case-insensitive email; valid role and active flag.
DR-02 Patients	patient_id, user_id, profile_details	Unique user_id references Users.
DR-03 Doctors	doctor_id, user_id, specialization, qualification, experience, fee, biography, image	Unique user_id references Users; specialty references Specializations.
DR-04 Specializations	name	Primary key name; referenced by Doctors.
DR-05 Availability	slot_id, doctor_id, date, start_time, end_time, status	Doctor FK; unique doctor/date/start; start before end; no overlapping doctor intervals.
DR-06 Appointments	appointment_id, patient_id, doctor_id, slot_id, date, time, status, created_at	Patient/doctor/slot FKs; one noncancelled appointment per slot.
DR-07 Payments	payment_id, appointment_id, amount, method, status, transaction_reference	Unique appointment and reference; simulation only; nonnegative amount.
DR-08 Notifications	notification_id, user_id, appointment_id, message, state, created_at	User FK; optional appointment FK; read/unread.
DR-09 Sessions	jti, user_id, expires_at	Session ID primary key; user FK; expiry required; not exportable business data.

3.3.1 Integrity and persistence

The system shall enable foreign-key enforcement and use transactional writes. Appointment date/time and doctor shall agree with the selected slot. A booked slot shall correspond to a noncancelled appointment. Each appointment shall have the matching successful simulated payment or refund state. Data shall survive normal application restart and schema reinitialization.

3.3.2 Confidentiality and backup boundaries

Password hashes, signing secrets, session tokens, and generated installation credentials shall not enter public reports or the exported workbook. Backups shall use SQLite’s consistent backup mechanism rather than an uncontrolled copy of a live database. Restoration remains an operator procedure; an automated scheduled backup service is not specified.

3.3.3 Formats

Clinic scheduling shall use Asia/Kolkata time. UI monetary values shall use INR. Identifiers shall remain stable across export/import and booking changes; generated business IDs need not expose sequential counts. Source-only distribution shall generate installation-local data and credentials instead of publishing an existing private database.

3.4 Nonfunctional Requirements

ID	Requirement / acceptance criterion	Evidence or status
NFR-01 Security	Passwords shall be stored as randomly salted scrypt hashes; signed sessions shall use HttpOnly/SameSite=Strict cookies, with Secure enabled in production.	Implemented; auth tests and configuration inspection.
NFR-02 Access control	Each protected request shall recheck account activity, session validity, role, and relevant record ownership.	Implemented; session/role/ownership tests.
NFR-03 Request safety	Parameterized SQL and request schemas shall handle user inputs; state-changing requests with an untrusted supplied Origin shall be rejected.	Implemented; invalid input and origin tests.
NFR-04 Authentication throttling	After 20 login/register attempts per client-host and endpoint in the preceding 60 seconds, further attempts shall return 429 until capacity clears.	Single-process in-memory implementation; not a distributed rate limiter.
NFR-05 Atomicity	Booking and rescheduling shall commit related changes together or leave the prior state unchanged.	Transactional implementation; conflict tests and prior fault-injection evidence.
NFR-06 Concurrent correctness	Eight competing valid requests for one available slot shall yield one booking and seven conflicts.	Verified scenario; not a general system-capacity claim.
NFR-07 Usability	Core interactions shall provide consistent labels, visible feedback, review/confirmation, and responsive navigation.	Browser checks and screenshots; no formal certification.
NFR-08 Portability	Provide a Bash launcher for supported Python/Node environments, preserving local data across normal restarts.	Linux launcher verified; native Mac/Windows execution not verified.
NFR-09 Configuration	Default local trusted origins shall follow PORT; explicit ALLOWED_ORIGINS shall take precedence and unrelated origins shall remain rejected.	Five configuration regression cases verified.
NFR-10 Maintainability	Separate UI, API, authentication, appointment, persistence, and workbook responsibilities; retain dependency locks and regression tests.	Repository structure and locked dependency files.

3.4.1 Targets requiring separate validation

Target ID	Proposed engineering target	Status
TARGET-01	Common interactions below three seconds	Not established as an SLA; workload, hardware, dataset, and percentile need an agreed benchmark.
TARGET-02	Support 200 concurrent users	Unverified; requires a representative workload and sustained load testing.
TARGET-03	99% service availability	Unverified; requires public hosting, an observation period, monitoring, and agreed exclusions.

These targets are not passed acceptance criteria. SQLite’s single-writer design, the single-process rate limiter, and hosting resources must be considered before adopting a production load or uptime commitment.

3.5 Business Rules and Error Behaviour

ID	Rule
BR-01	Only active doctors and strictly future available slots shall be publicly bookable.
BR-02	There shall be no more than one noncancelled appointment for a slot. Cancellation releases the slot; completion does not.
BR-03	Through the appointment-change APIs, cancellation/rescheduling shall require a confirmed appointment and at least two hours before its current start. Exactly two hours is eligible; less than two hours is rejected. The implemented rule also applies to administrator changes.
BR-04	Rescheduling shall stay with the same doctor, reserve a valid new slot, release the old one atomically, and keep the existing payment relationship.
BR-05	Only the assigned doctor or an administrator shall complete a confirmed appointment, and not before its scheduled start.
BR-06	Only simulated payments/refunds shall be recorded. A failed simulated payment shall not reserve a slot or create a confirmed appointment.
BR-07	Doctor availability shall not overlap; an already booked slot shall not be removed.
BR-08	In-app notifications belong to their addressed user; reminders shall not imply an external scheduler or message delivery.
BR-09	SQLite is authoritative. Excel preview shall not commit, and Excel import shall not delete records merely because rows are absent.
BR-10	Custom names and later administrator edits shall survive normal demo restarts; production/demo-disabled configurations shall not run demo identity migrations.

3.5.1 Failure handling

Validation or rule errors shall return an explanatory response without partial business writes. A slot conflict shall invite selection of another slot. Incorrect login responses shall not confirm whether the email exists. Unexpected persistence failures shall roll back the current transaction; recovery requires fixing the underlying problem and retrying, rather than deleting the database.

3.5.2 Appointment change examples

Situation	Required outcome
Confirmed consultation starts in 3 hours; patient cancels	Cancel; release slot; record simulated refund.
Confirmed consultation starts in exactly 2 hours; patient reschedules	Allow if the new slot satisfies all other rules.
Consultation starts in 90 minutes; cancellation requested	Reject; retain original records.
Patient reschedules to another doctor	Reject; retain original appointment.
Doctor completes before the start time	Reject; retain confirmed state.

Appendix A. Core Use Cases

UC-01 Authenticate a user

Field	Specification
Primary actor	Patient, doctor, or administrator
Preconditions	Account exists, is active, and login is not currently throttled.
Trigger / inputs	Submit email and password.
Main flow	Validate input → find account → verify hash and activity → create session and signed cookie → retrieve account role → display role dashboard.
Postcondition	A usable, revocable authenticated session exists.
Alternatives	Malformed input: 422; invalid/inactive credentials: generic 401; attempt limit: 429; supplied untrusted Origin: 403.
Related requirements	FR-02–FR-04; NFR-01–NFR-04; NFR-09

UC-02 Book a consultation

Field	Specification
Primary actor	Authenticated patient
Preconditions	Active doctor and future available slot; fictional fee displayed.
Trigger / inputs	Select slot and submit simulated payment outcome after review.
Main flow	Authorize patient → begin transaction → recheck slot → process simulation → create confirmed appointment/payment → mark slot booked → notify patient/doctor → commit → show confirmation.
Postcondition	One consistent booking exists with matching payment and slot state.
Alternatives	Missing slot 404; occupied slot 409; past/equal-time slot 422; failed simulation 402; unauthenticated 401 or wrong role 403. Any failed transaction leaves no partial booking.
Related requirements	FR-08–FR-12; NFR-05–NFR-06; BR-01–BR-02; BR-06

UC-03 Change a booking

The owning patient or administrator selects a confirmed appointment. The system verifies ownership, role, and the two-hour cutoff. Cancellation refunds and releases the slot; rescheduling validates a same-doctor future slot and changes both slot states atomically. Ineligible or conflicting requests retain the previous booking. Related: FR-13–FR-14 and BR-03–BR-04.

UC-04 Exchange appointment records with Excel

An administrator exports a fresh workbook, edits permitted fields while retaining schema and IDs, validates/inspects the preview, and applies import. The system repeats validation against current data before commit. Invalid data rolls back and receives a useful message. Related: FR-24–FR-26 and BR-09.

Appendix B. Acceptance and Traceability

The following matrix maps requirements to implementation and verification evidence. This documentation task did not rerun the software; the recorded results are the completed Experiment 9/10 runs, not a new certification of every requirement.

Requirement group	Implementation	Acceptance evidence
FR-01–FR-05; NFR-01–NFR-04	schemas.py, security.py, auth routes, safeguards	Registration/hash/cookie/logout, roles, profile, and invalid-input tests; rate-limit threshold also established by source inspection.
FR-06–FR-08; UI-01–UI-03	Doctor routes and React listing/profile pages	Filter/sort checks, doctor availability tests, browser desktop/mobile workflows.
FR-09–FR-14; BR-01–BR-04; BR-06	appointments.py, transactional database layer	Booking, failed payment/retry, conflict, persistence, cancellation/refund, reschedule and cutoff tests.
FR-15–FR-21; BR-05–BR-07	Doctor/admin routes and dashboards	Ownership, slot overlap/removal/completion, management, account/session and report checks.
FR-22–FR-23; BR-08	Notification routes and notifications table	Notification ownership/read checks; reminder condition and deduplication inspected in source.
FR-24–FR-26; BR-09	excel.py and admin workbook UI	Round-trip, preview/import, integrity rollback, formula, limits, and browser exchange tests.
DR-01–DR-09; NFR-05–NFR-06	SQLite schema, indexes and transactions	FK/unique constraints; one winner in eight requests; prior injected rollback and restart-persistence checks.
NFR-08–NFR-09	start.sh and PORT-derived origins	Actual launcher, build/health checks, and five isolated configuration cases.

Recorded test results

Run	Outcome / boundary
Experiment 9 focused suite	23 passed; includes authentication, booking, time boundaries, fault injection, and a password-verification unit test.
Experiment 9 existing suite	21 passed; overlaps with the later regression suite and is not an additional unique-test count.
Experiment 10 full backend suite	26 passed: the 21 existing workflow tests plus 5 origin-configuration regression cases.
Experiment 10 browser smoke checks	4 scenarios passed: failed login, valid login, failed payment, successful retry/confirmation.
Deployment/build	TypeScript/Vite production build, Bash launcher, API health and compiled frontend serving verified in Linux.

Source inspection supports requirements not individually asserted in the named runs. Unperformed load, availability, native-platform, and full accessibility checks shall remain explicitly unverified. Faculty review shall confirm requirement scope and any additional acceptance work before treating the document as an approved baseline.

Appendix C. Feasibility and Requirement Reconciliation

C.1 Feasibility assessment

Dimension	Assessment
Technical	Existing React/FastAPI/SQLite stack implements the selected academic workflows; higher concurrency would require further architecture and load evaluation.
Economic	Open-source runtime and local demonstration avoid mandatory paid services; hosting and future integrations may incur costs.
Operational	Role-specific workflows and administrator controls support a supervised demonstration; users need credentials and basic browser/Excel skills.
Schedule	Requirements, diagrams, implementation, test design, correction, and local deployment have been produced incrementally. No independently audited effort estimate is claimed.

C.2 Ambiguities and inconsistencies resolved

Earlier wording / ambiguity	Baseline decision
MySQL suggestions versus explicit SQLite/Excel preference	Use SQLite; Excel is validated exchange, not a second live datastore.
Password encryption	Use salted one-way hashing, not recoverable password encryption.
Payment and notification wording	Simulated payment and in-app messages; real gateways and external delivery remain excluded.
Appointment workflow versus stored states	Persist confirmed/cancelled/completed; review and payment attempts are transient steps.
Broader hospital/library text in earlier experiments	Retain appointment-system scope; unrelated library conclusion and optional clinical modules do not add requirements.
Custom-port startup	Generate default local origins from PORT while preserving explicit ALLOWED_ORIGINS.

C.3 Development and change plan

Maintain the sequence: requirements baseline → DFD/use-case/class/state/activity models → API/data/UI implementation → requirement-linked tests → reproduce and correct defects → rebuild and locally deploy → maintain source and evidence. Future changes shall identify affected requirement IDs, update tests, and record a new document version. Faculty approval, public deployment, and acceptance of performance targets remain separate decisions.

C.4 References

- SE-Lab Manual.docx: Experiment 1 requirements task and Part B SRS question; Experiments 9 and 10 testing/correction tasks.
- Project REQUIREMENTS_TRACEABILITY.md, README.md, docs/ARCHITECTURE.md, docs/DEPLOYMENT.md, and source at repository reference 89168e0.
- Experiment 9 report and test evidence; Experiment 10 report and before/after configuration evidence.
- IEEE 830-style document organization is used as a structural convention; no formal conformance assessment or third-party template download is claimed.